API SECURITY RISK ANALYSIS REPORT

PREPARED BY: ANIL KASHYAP

DATE: AUGUST 30, 2026

TARGET SCOPE: REQRES API TESTING

PLATFORM (HTTPS://REQRES.IN)

- the.anilkashyap01@gmail.com
- GitHub

1. EXECUTIVE SUMMARY

A READ-ONLY SECURITY ASSESSMENT WAS CONDUCTED AGAINST THE REQRES DEMO API TO IDENTIFY ARCHITECTURAL SECURITY FLAWS ALIGNING WITH THE OWASP API SECURITY TOP 10. THE ANALYSIS FOCUSED ON ENDPOINT EXPOSURE, DATA FORMATTING, AND RATE-LIMITING CONTROLS. THE ASSESSMENT REVEALED VULNERABILITIES RELATED TO UNAUTHENTICATED DATA ACCESS AND EXCESSIVE DATA EXPOSURE (EXPOSING PII), THOUGH THE PLATFORM DEMONSTRATED ROBUST RATE-LIMITING CONTROLS.

2. SCOPE & METHODOLOGY

METHODOLOGY:

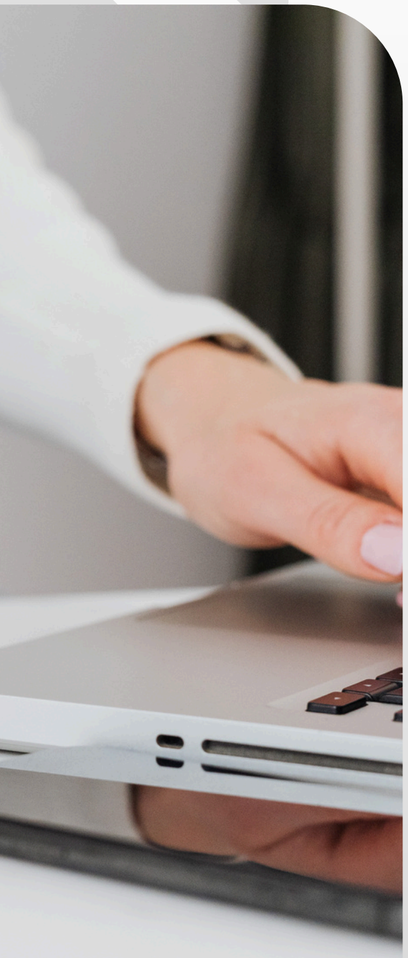
Black-box, read-only API inspection.

TOOLS UTILIZED:

Postman (HTTP Client) for endpoint querying and response validation; Browser DevTools for header inspection.

CONSTRAINTS:

No automated exploitation or denial-of-service (DoS) attacks were performed.



3. IDENTIFIED API SECURITY RISKS

RISK 1: OPEN / UNAUTHENTICATED ENDPOINT (BROKEN AUTHENTICATION)

- Severity: HIGH
- Endpoint Tested: GET /api/users/2
- Observation: The API endpoint was successfully queried without providing any authentication headers (e.g., Bearer Token, API Key, or Session Cookie). The server responded with an HTTP 200 OK status.
- Business Impact: In a production SaaS environment, leaving user data endpoints unauthenticated allows malicious actors to access, scrape, and exfiltrate the entire user database anonymously, leading to severe privacy breaches and regulatory fines.
- Remediation Suggestion: Implement strict Authentication controls (e.g., OAuth 2.0 or JWT). Ensure the API gateway drops requests lacking a valid Authorization header, returning an HTTP 401 Unauthorized status.

RISK 2: EXCESSIVE DATA EXPOSURE (PII LEAKAGE)

- Severity: MEDIUM / HIGH
- Endpoint Tested: GET /api/users/2
- Observation: The JSON response payload returned complete user objects, including id, email, first_name, last_name, and avatar URL links.
- Business Impact: APIs often return all available database fields for a user, assuming the front-end application will filter what the user actually sees. Attackers intercepting the raw API traffic can view sensitive fields (like emails or internal IDs) that were not intended for public display.
- Remediation Suggestion: Implement strict data filtering at the API layer (not the client level). Use GraphQL or specific REST query parameters (e.g., ?fields=first_name,last_name) to ensure the API only returns the exact data necessary for the UI component being rendered.

RISK 3: RATE LIMITING & RESOURCE EXHAUSTION CONTROLS

- Severity: LOW (Mitigated / Controlled)
- Endpoint Tested: GET /api/users (Rapid succession testing)
- Observation: The testing successfully triggered the server's rate-limiting defenses. The server responded with an HTTP 429 Too Many Requests status code and a JSON body stating "error": "rate_limit_exceeded".
- Business Impact: Because the API correctly limits unauthenticated requests to 40 per day per IP, it is highly resilient against automated scraping, brute-force attacks, and Denial of Service (DoS) attempts.
- Remediation Suggestion: The current implementation is secure. To further enhance visibility, ensure that 429 errors are actively logged and monitored by the SOC team to identify aggressive IP addresses.

4. SECURITY HEADERS ANALYSIS

Passive inspection of the HTTP Response Headers revealed several positive security configurations:

- X-Content-Type-Options: nosniff: Prevents MIME-sniffing attacks.
- X-Frame-Options: DENY: Protects against Clickjacking attacks.
- Strict-Transport-Security: max-age=31536000; includeSubDomains: Enforces secure HTTPS connections.

THANK YOU